Algorithms PA 2 Report

B13901186 謝秉宸

Introduction

Given a set C of N chords in a circle, the points are labeled from 0 to $2N-1$. The objective is to find the maximum subset of chords that does not intersect each other, i.e. the maximum planar subset. Denote $M(i,j),\ i \le j$ to be the number of chords in the maximum planar subset in the region formed by chord $\overline{ij}$ and the arc ij. In HW2, the following recurrence relation is found:

Let k be the other endpoint of chord $\overline{jk}$

Case 1: $i = j$,

$$M(i,j) = 0$$

Case 2: $i = k$,

$$M(i,j) = M(i+1, j-1) + 1$$

Case 3: $k < i \ or \ k > j$,

$$M(i,j) = M(i, j-1)$$

Case 4: $k > i \ and \ k < j$,

$$M(i,j) = max(M(i,k-1) + M(k+1, j-1) + 1, M(i, j-1))$$

where $M(i,\ k-1) + M(k+1,\ j-1) + 1$ is the case when $\overline{jk}$ is included, and $M(i,\ j-1)$ is the case when $\overline{jk}$ is excluded.

Below I will demonstrate a time and memory-efficient algorithm to solve the problem.

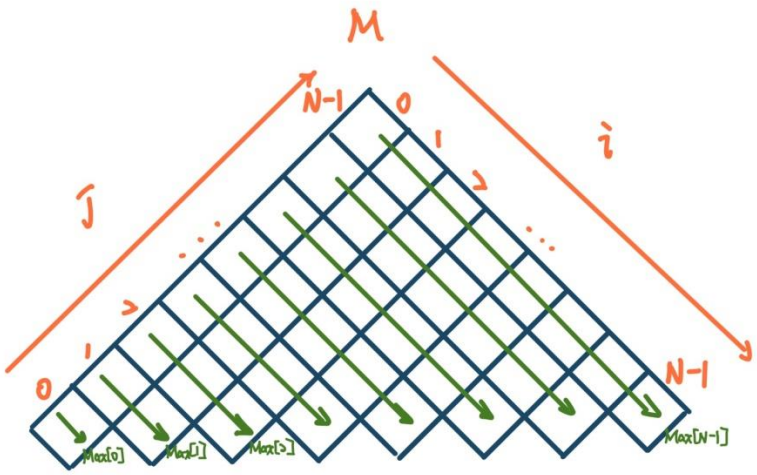
Data Structure

First, a **int* chords** array of length is used to store the endpoints of each chord. For every chord $\overline{ij}$, **chords[i]=j** and **chords[j]=i**.

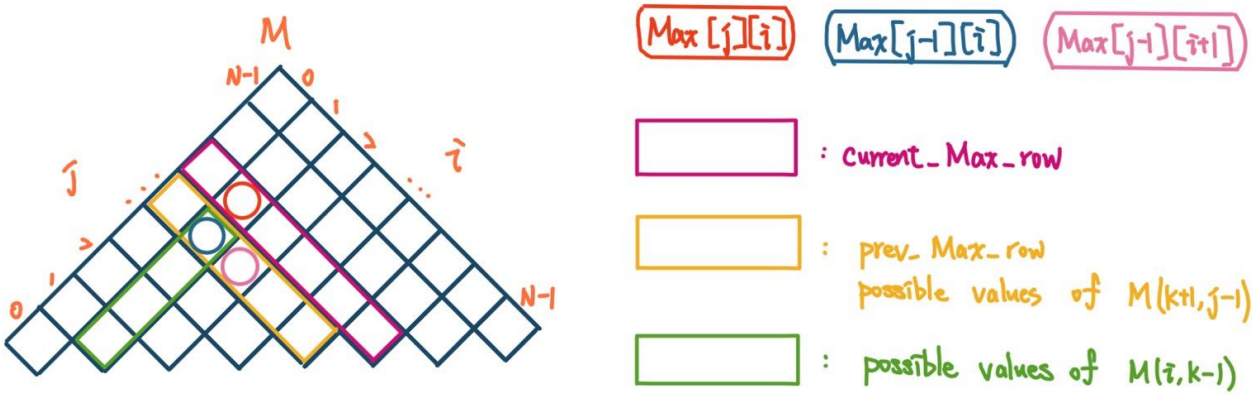
Second, two sets of data structure storing the length of the maximum planar subset are utilized and compared.

1. 2-dimensional array **int** Max**:

For the j-th index ($0 \leq j < 2N$) of **Max**, a pointer which points to an array with length $j + 1$ is stored in **Max[j]**. Store $M(i, j)$ in **Max[j][i]**. It can also be used to reconstruct the output. The relationship between M and **Max** is shown below:



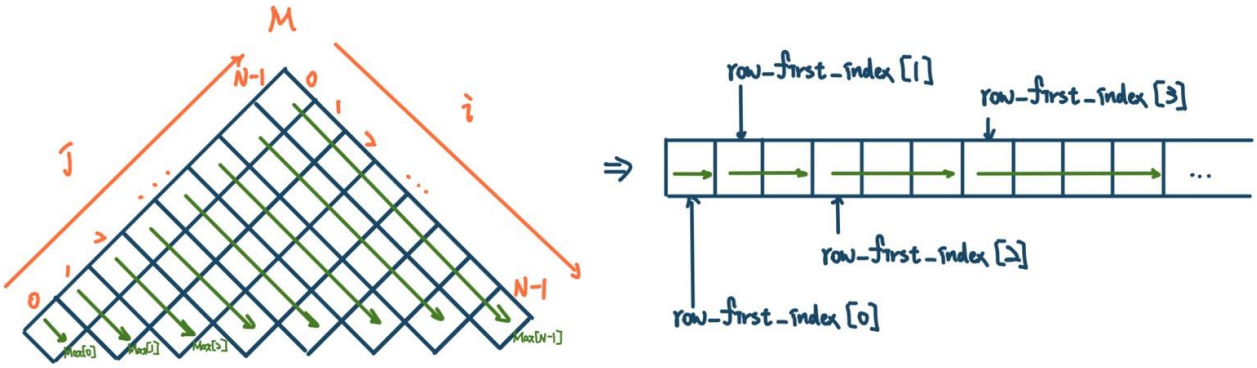
There are various benefits if we store the elements this way. First, as I will mention in the algorithm part below, when we iterate over each j, the address of **Max[j]** and **Max[j-1]** will be stored as **int* curr_Max_row** and **int* prev_Max_row** respectively. By storing these two addresses, when we iterate through i for each j, we can find almost all of the values required to update **Max[j][i]** efficiently. The reason is that they are located near these two addresses, which will be illustrated below. In ***Case 1***, we can easily update **Max[j][i]** by finding **prev_Max_row[i + 1]**. Similarly, in ***Case 2***, we can find **prev_Max_row[i]**. In ***Case 3***, if chord $\overline{jk}$ is excluded, we can find the value in **prev_Max_row[i]**. If chord $\overline{jk}$ is included, we need to find $M(i,\ k-1)$ and $M(k+1,\ j-1)$. For the first term, we need to find **Max[k-1][i]**, which can be very time-consuming if N is large (e.g. 90000). But luckily, for the second term, we can locate it in **prev_Max_row[k + 1]** which can be found quickly.



2. 1-dimensional array **int* Max** + 1-dimensional array **size_t* row_first_index**:

The 2D dynamic array is compressed to a 1D dynamic array with length $\frac{2N\times(2N+1)}{2}$. Another 1D array is needed to store the first index of each "row".

Since N may be as large as 90000, I use the integer type **size_t** to store the index values while minimizing the required space. Thus, $M(i,j)$ is stored in **Max[row_first_index[j] + i]**. The compression from 2D to 1D is shown in the figure below:



Last, a **vector<int> result** array is used to store each smaller endpoint of the output chords. The length of the vector is not initialized because it depends on the input chords.

Algorithm

The source code is provided in ./src/main.cpp. There are 4 major steps in the algorithm: initialization, compute MPS, reconstruct output, and ending.

- Step 1: Initialization

  Use **fstream** to read the input and write the output. Then, initialize the following arrays.

  int chords = new int [2N];*

  int Max = new int *[2N*(2N+1)/2]; //For 1D array case*
  size_t row_first_index = new size_t [N+1]; //For 1D array case*
  *int** Max = new int *[2N]; //For 2D array case*

  vector<int> result;

- Step 2: Compute MPS

  In the function **compute_mps()**, the 2D array **Max** is filled using the recurrence defined in the introduction.

  For the 1D array data structure, **compute_mps()** takes in **Max**, **chords**, **row_first_index** and **num_pts**($= 2N$) as input variables. For the 2D array data structure, **compute_mps()** takes in **Max**, **chords** and **num_pts**($= 2N$) as input variables.

  Because every sub-problem needs to be solved to find the solution, I use the "bottom-up" approach. This way, stack overflow can also be prevented when dealing with large input data.

  When filling the array, the j index is iterated first. For every value of j, the address of **Max[j]** and **Max[j-1]** is recorded and saved as **curr_Max_row** and **prev_Max_row**. Then for every value of j, the i index is iterated from 1 to $j-1$. For every pair of (i, j), the recurrence relationship is applied to calculate **Max[j][i]**.

  Case 1: If $i = j$, **Max[j][j]** is set to 0.
  Case 2: If $i = k$, **Max[j][i]** is set to **prev_Max_row[i + 1]** + 1.
  Case 3: If k $>$ j or k $<$ i, **Max[j][i]** is set to **prev_Max_row[i]**.
  Case 4: Otherwise, compare the two values:
  When chord $\overline{jk}$ is included, the value (**inc_jk**) is **Max[k - 1][i]** + **prev_Max_row[k + 1]** + 1.
  When chord $\overline{jk}$ is excluded, the value (**not_inc_jk**) is **prev_Max_row[i]**.
  Select the maximum of the two and assign it to **Max[j][i]**.

  We need to be careful in some special cases. In ***Case 2***, there is one specific case when $i = k = j - 1$, the original algorithm will assign $M(i+1, j-1) + 1 = M(j, j-1) + 1$ to **Max[j][i]**, which is out of bound. So in this case, **Max[j][i]** is set to 1.

  In ***Case 4***, when $k = j - 1$, the $M(k+1, j-1)$ term becomes $M(j, j-1)$, which is again out of bound. So in this case, set $M(k+1, j-1)$ to 0.

- Step 3: Reconstruct Output

  The function **print_mps_chords()** to find all the chords in the maximum planar subset in increasing order.

  For the 1D array data structure, **print_mps_chords()** takes in **row_first_index, chords**, **result**, **i**, and **j** as input variables. For the 2D array

data structure, **print_mps_chords()** takes in **Max, chords**, **result**, **i**, and **j** as input variables.

Instead of initializing another array to record the case number of $M(i,j)$, I used the original table array **Max** to reconstruct the output. Similar to Step 2, for selected pairs of (i,j), determine the case number and call the function recursively.

Case 1: If $i = j$, do nothing and return.
Case 2: If $i = k$, first add the value i to **result**,
then call **print_mps_chords(…, $i+1$, $j-1$)** .
Case 3: If k > j or k < i, call **print_mps_chords(…, i, $j-1$)**.
Case 4: Otherwise, compare the two values with **Max[j][i]**:
When chord $\overline{jk}$ is included, the value (**inc_jk**) is **Max[k - 1][i]** + **prev_Max_row[k + 1]** + 1.
When chord $\overline{jk}$ is excluded, the value (**not_inc_jk**) is **prev_Max_row[i]**.

If **inc_jk = Max[j][i]**, call **print_mps_chords(…, i, $k-1$)**, then add the value k to **result**, finally call **print_mps_chords(…, $k+1$, $j-1$).**
Otherwise, call **print_mps_chords(…, i, $j-1$)**

Again, we need to be careful when $k = j - 1$ and set all the values of out-of-bound indices to 0.

- Step 4: Ending
  Close **fstream** and delete the dynamic arrays to free up memory. Calculate the total runtime and observe the memory usage.

Results

Here I compare the results of the 2 sets of different data structures. First, record the runtime for $N = 6, 500, 5000, 50000$ and observe the memory usage for $N = 50000$. Since both time and space complexity of the maximum planar subset problem is $O(N^2)$, we can estimate the runtime and memory usage for $N = 90000$. (Run on port 40060 of EDAUnion)

- 1D array case

N	6	500	5000	50000		
	Time(ms)	Time(ms)	Time(ms)	Time(ms)		Memory(GB)
Calculate MPS	<1	8	333	31070		18.47
Reconstruct	<1	<1	0	8		
Overall	4	14	348	31972		

For $N = 90000$,

Estimated runtime: $31.972 \times \left(\frac{9}{5}\right)^2 = 103.59$(s)

Estimated memory usage: $18.47 \times \left(\frac{9}{5}\right)^2 = 59.84$(GB)

- 2D array case

N	6	50	5000	50000		
	Time(ms)	Time(ms)	Time(ms)	Time(ms)		Memory(GB)
Calculate MPS	<1	7	316	30136		18.32
Reconstruct	<1	<1	0	8		
Overall	4	11	336	31304		

For $N = 90000$,

Estimated runtime: $31.304 \times \left(\frac{9}{5}\right)^2 = 101.42$(s)

Estimated memory usage: $18.32 \times \left(\frac{9}{5}\right)^2 = 59.36$(GB)

Discussion

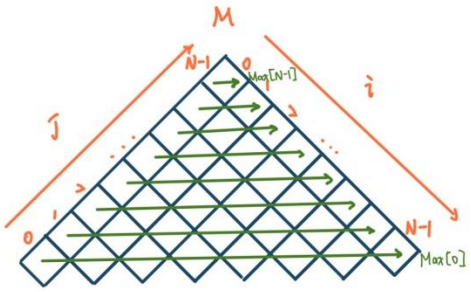
From the tables above, we can see that most of the runtime is spent on calculating the maximum planar subset (filling in the array), while the time spent on reconstructing the output is nearly negligible. On the other hand, we can observe that both the runtime and memory usage for the 1D and 2D array case are very similar.

In terms of time complexity, using a 1D array guarantees that the data is contiguous in memory. However, indexing the array still takes time because **row_first_index[j] + i** needs to be computed, which can be a very large number when N is large. When using a 2D array, the memory location of each row is not contiguous, but indexing is simpler since it only requires two pointer dereferences.

In terms of space complexity, using a 1D array requires an additional array to

record the index of each row. This is similar to the 2D array case where the memory address of the j-th row is stored in **Max[j]**.

Finally, using the optimal way of storing each element in the array, the program can index the array efficiently to find the values needed for recursion. If other storage methods are used, the runtime will be significantly increased. The picture below illustrates one example:



Conclusion

Using a suitable algorithm and an optimal data structure, I found a time and space efficient way of solving the maximum planar subset. For the largest input data given ($N = 50000$), a runtime of 31.304(s) and 18.32(GB) of memory usage are achieved using a 2D array. For the largest possible testing data ($N = 90000$), the fastest runtime estimated is 101.42(s) and the corresponding memory usage is 59.36(GB).